两次解耦，把 Coding Agent 带进日常应用



谢威宇

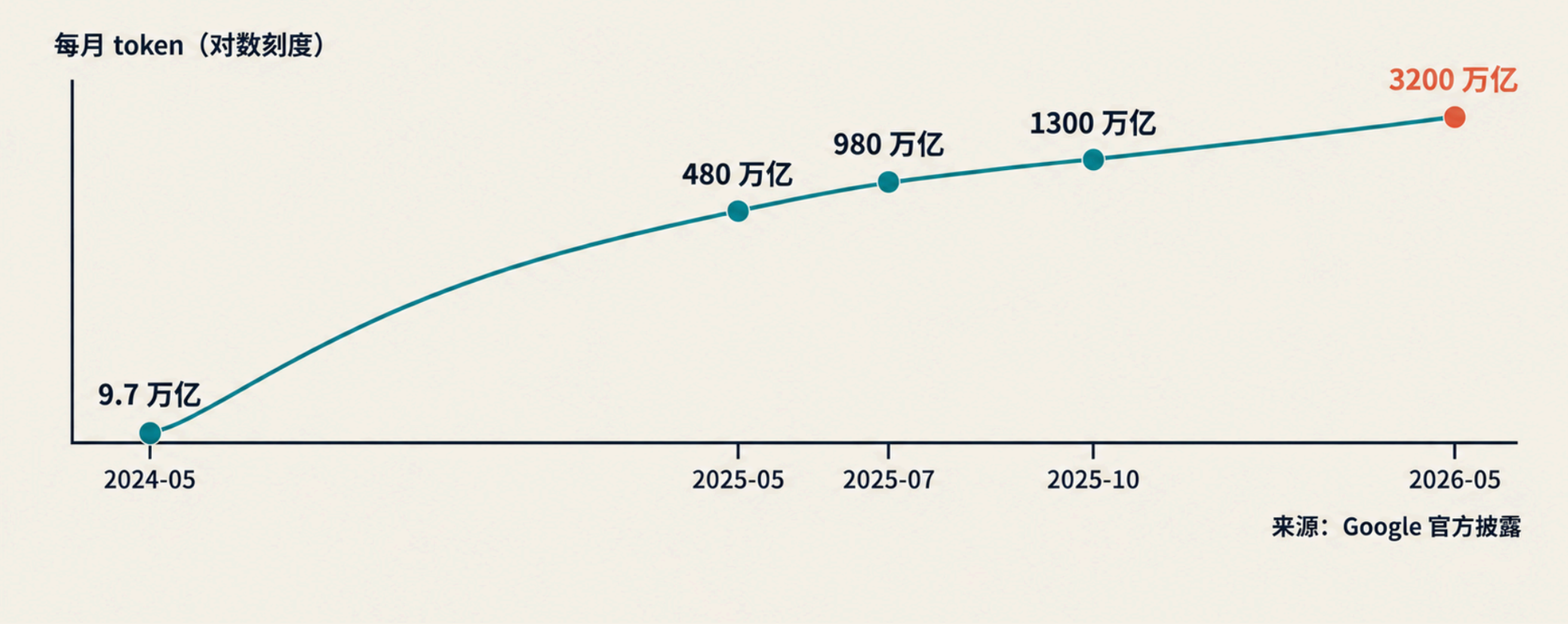
趋境科技首席架构师

SuperMe 实践

2026.10

模型消耗的 token，两年里涨了 330 倍

Google 公布的全平台月度 token 处理量，两年里从 9.7 万亿涨到 3,200 万亿；2022–2023 年没有同口径的官方披露。



① 使用者的规模扩大

越来越多的个人与组织开始调用模型

② 单个使用者的消耗上升

同一个人对模型的使用深度持续增加

人的工作时间有上限，所以比例必须上升

一天 24 小时里，只有「工作」这一段在调用注意力，而这一段的时间无法无限增长。



因此人均 token 要增长，只能是模型 token 与人类 token 的比例上升。

从 Chat 到 Agent 集群，人的占比一路下降

同一件事，交给越自主的形式，人需要投入的注意力越少，模型能消耗的 token 越多。

Chat

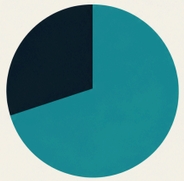




人 50%　AI 50%

Deep Research

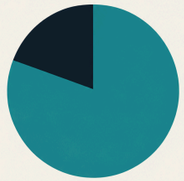




人 30%　AI 70%

Coding Agent

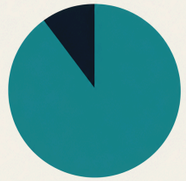




人 20%　AI 80%

Goal Agent





人 10%　AI 90%

大规模 Agent 集群





人 1%　AI 99%

饼图为示意：表示人类介入的相对比例，不是实测统计。

人占的比例越小，模型能消耗的 token 越多——这就是人均 token 的增长空间。

能烧掉大量 token 的任务：可判断，且足够难

能不能吃下大量 token，取决于这件事的价值能不能被判断、以及它够不够难。



关键不是把 Agent 部署到更多地方，而是让更多任务变得可以判断。

两次解耦，留下的是同一个 Context

解耦的意思是把原本绑在一起的两件事分开——两次解耦的方向不同，但留下的都是同一份 Context。

第一次解耦	**单个 Agent session 与 Context 分开。**
第二次解耦	**Agent Runtime 所在的设备与 Context 分开。**

至于 Agent Context 到底是什么、为什么它必须独立于一次运行，后面几页会展开。

> **把 Agent Context 这个核心留下，其他逐步剥离开。**

01 / FIRST DECOUPLING

第一次解耦

把单个 Agent session 与 Context 分开。

工作状态不再属于某一次运行。

Agent Context 就是一个 repo

一个目录，Coding Agent 启动时先读它，各种 context 文本都存在里面；它由 git 管理，可以交接、可以持久化。

```
agent-context/
├── AGENT-RUNTIME.md   启动入口
├── meta-log/   过程，只追加
├── goals/   目标与状态
├── doc/   稳定认知
│   └── sop/   标准流程
└── scripts/   确定性动作

不是按目录名组织，
而是按"要对齐谁、
能稳定多久"来分层。
```

组织方式：越稳定越往上

```
静态的上下文 = 仓库
  一直累积，可以交接

动态的运行时 = 模型 + agent 程序
  可以随时替换

meta-log/   记过程，只追加不修改
doc/        沉淀稳定的认知
sop/        起始条件、结束条件、步骤
启动不必全读，按当前任务按需查阅

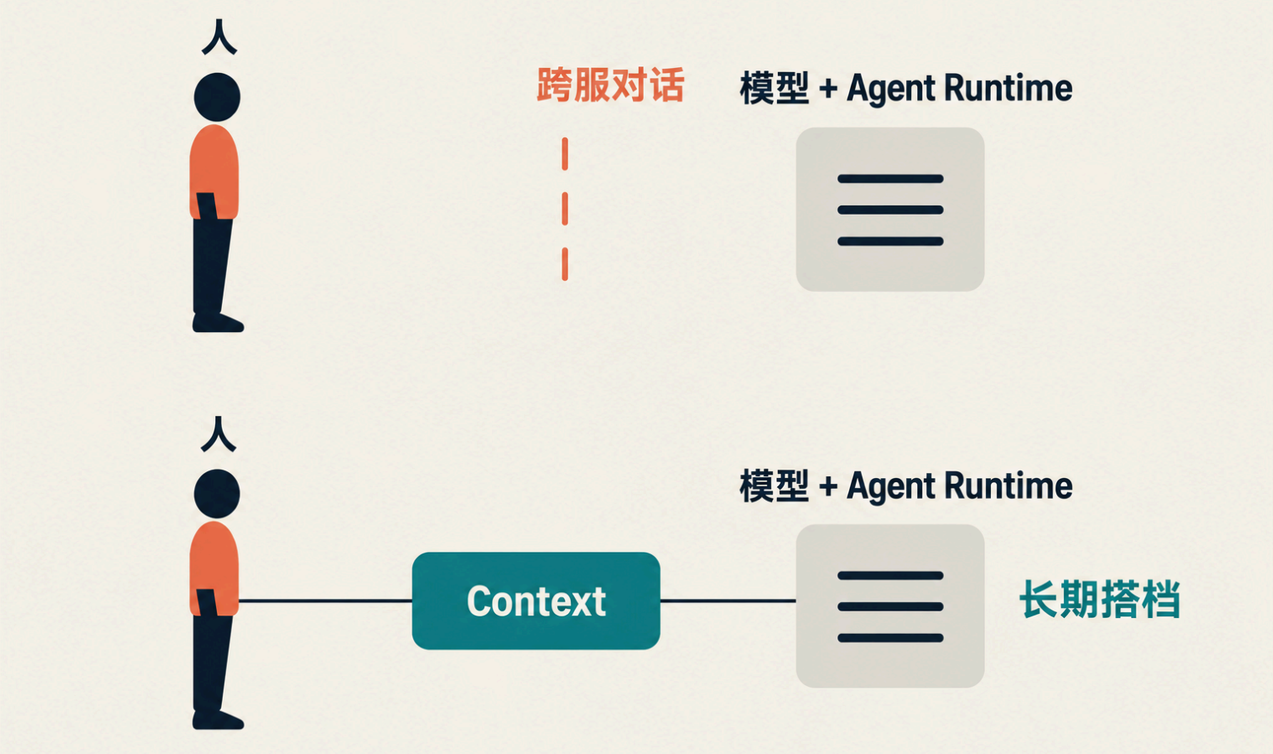
_____________________________

开源实现（含方法论与脚本）
github.com/Approaching-AI/meta-agent
```

仓库是静态的上下文，模型和 agent 程序只是可替换的运行时。

没有 Context 的人机交互，是跨服对话

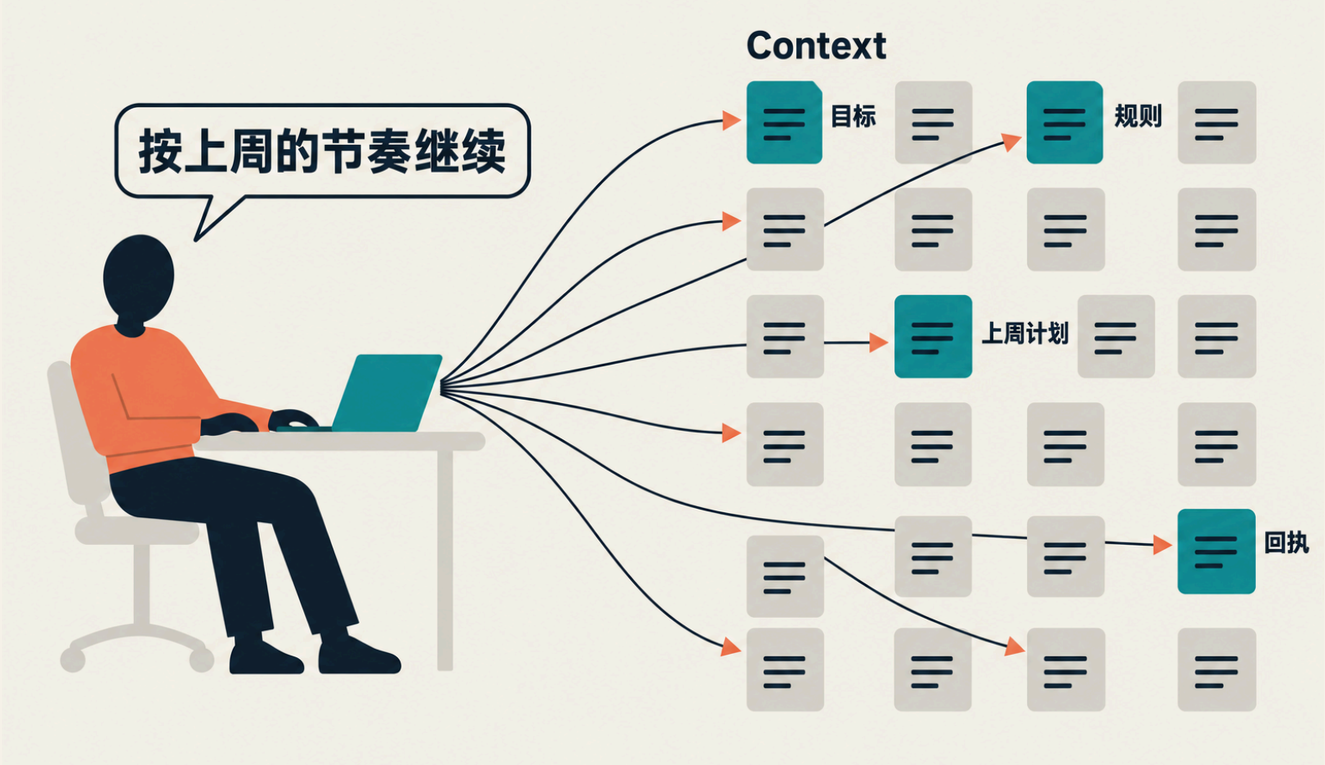
没有那个仓库，双方就看不到彼此的状态，判断成本也永远降不下来。



长期搭档效率高，不是因为谁更强，而是共享的背景把沟通压到了极短。

Context 是人机对齐的载体

有了那份共享的文本，人只需要说一句话，剩下的由 Agent 自己去读。



人给一句话，Agent 自己去读背景里剩下的部分。

一项长期的工作，不能指望一个 session 做完

工作越长期、越困难，就越跨越多轮运行；而 Runtime 会结束、会重启、会被替换。

用户目标 → **Runtime A** ↔ **持久 Context** ↔ **Runtime B**

可恢复

运行结束 / 进程中断 / 换 Runtime
→ 从已有状态继续

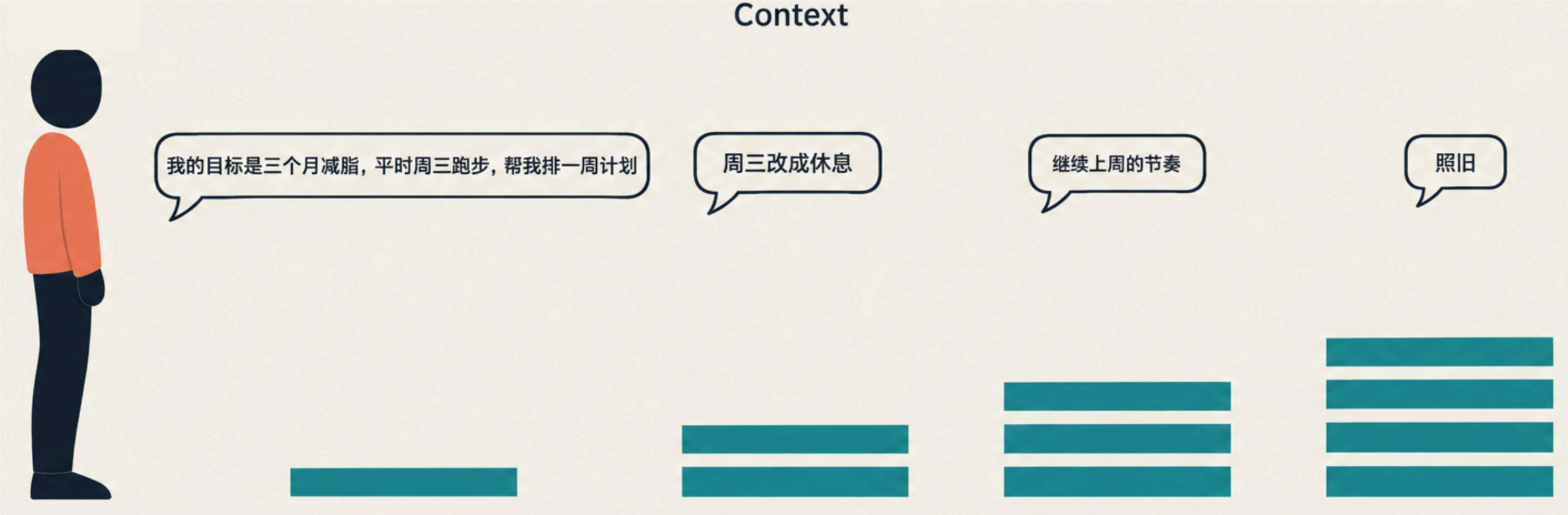
可累积

每轮把目标、决策、进度写回 Context
→ 下一轮从更高处开始，
不用把背景重讲一遍

可恢复让一次工作不被打断，可累积让对齐越用越省。

对齐不是一次配好的，是攒出来的

每轮都把目标、决策和进度写回 Context，背景越积越厚，人需要解释的越来越少。



用得越久，需要解释的越少——这就是 Context 能对齐人和 LLM 的原因。

Context 不是代码仓库，也不是代码文档

它们长得很像，但装的东西、服务的目的都不一样。

代码仓库

装的是代码

- 按代码边界切分
- 一个项目可以有多个
- 项目背景与设计取舍不在里面

代码文档

为了解释代码

- 读者是人，对象是代码
- 跟着代码走
- 说明的是实现，不是意图的来龙去脉

Agent Context

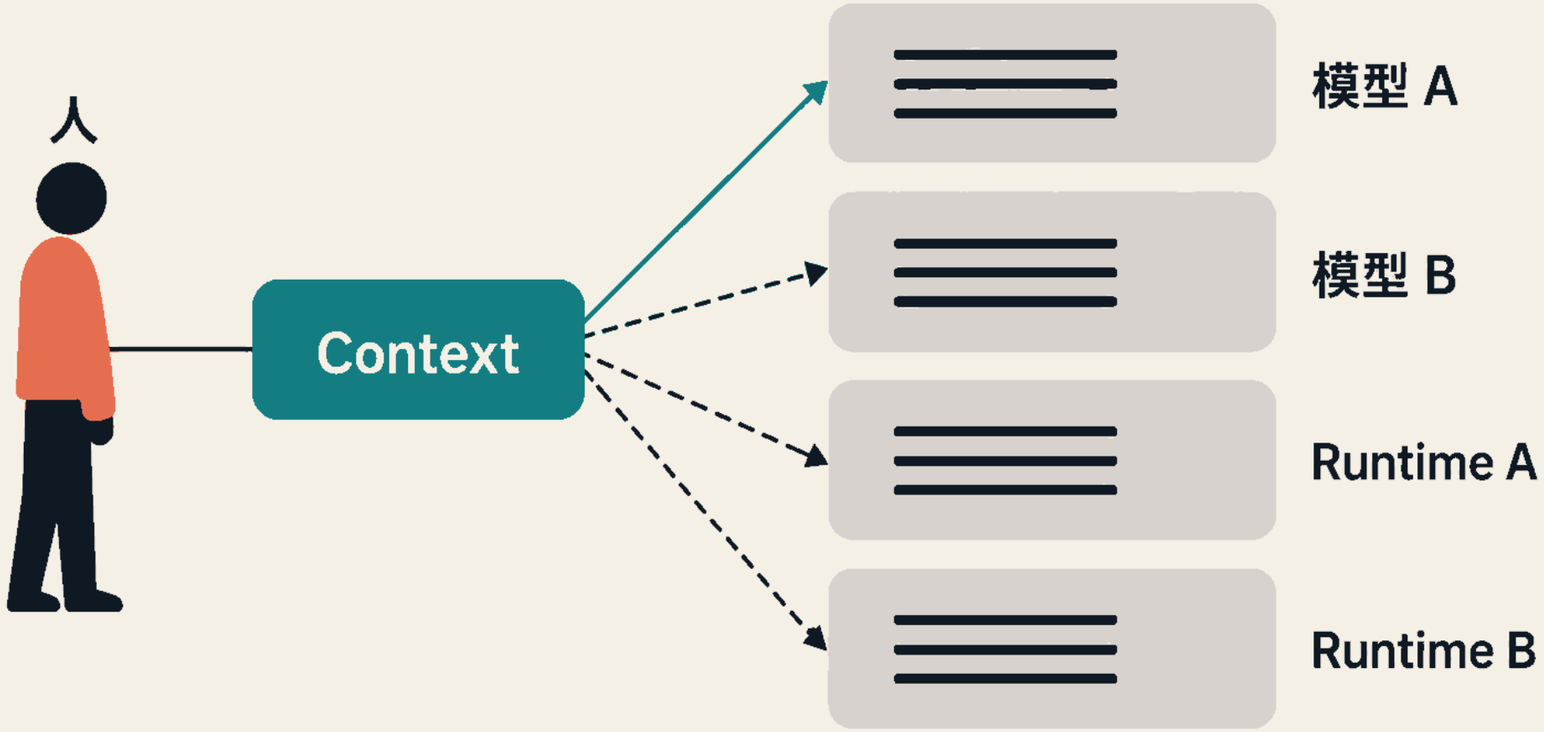
为了对齐人和模型

- 装目标、规则、约定、取舍与意图
- 按“要对齐谁”切分
- 多仓库共用一个，也可团队共用或分开

仓库按代码边界切分，Context 按“要对齐谁”切分。

Context 解除对单一模型供应商的依赖

Context 独立于运行、独立于载体，也就独立于跑它的那个模型。



权威状态在谁手里，从一句所有权主张，变成了可以直接看到的结果。

02 / SECOND DECOUPLING

第二次解耦

把 Agent Runtime 所在的设备与 Context 分开。

运行位置不再绑定在用户手上的那一台设备。

强大的 Agent，仍然被运行位置限制

Context 是静态文本，可以低成本贴近数据；Runtime 重的是 CPU——它要一直占用核心，而推理算力由 token 工厂承担。

CONTEXT · 轻

可以留在数据旁边

- 静态文本
- 随用户走
- 低成本保存

RUNTIME · 重

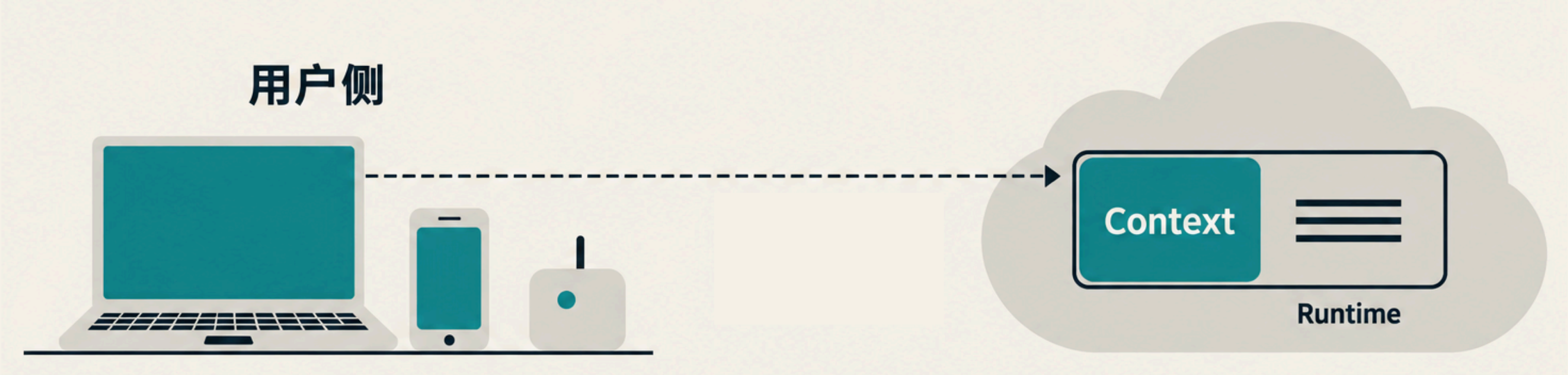
要一直占用 CPU 核心

- 调度与工具循环持续在跑
- 要一直在线
- 不适合常驻手机

能力被运行位置限制：想让它到处都在，就不能要求它到处都跑。

把 Runtime 搬上云，Context 也跟着一起上云

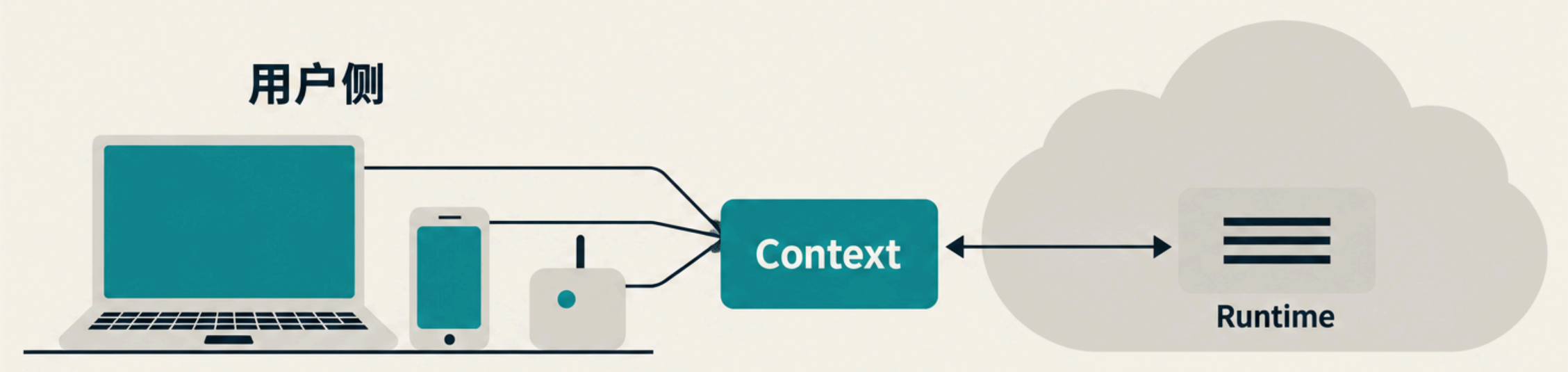
Runtime 跑不动的地方，默认解法是搬到服务器上；但连着一起搬的还有桌面上的工作环境，Context 于是又和 Runtime 绑在了一起。



例：Dots（OpenAI）、Muse（Meta）、Manus、Devin、cdxm server——都停在第一次解耦。

把 Agent Runtime 与 Context 解耦

云上只放运行能力；Context、应用和设备留在用户侧，两边只交换必要的状态与证据。



不托管用户的工作环境，只把 Agent 的运行能力作为服务提供。

一次 AaaS 任务如何完成

服务端负责推进工作，客户端负责提供事实、执行操作并保存结果。

01 读取 Context → 02 选择工具 → 03 调用客户端 → 04 本地执行 → 05 返回回执 → 06 写回状态

认证与配额

协议版本协商

流式进度

任务与尝试 · 可恢复

开源实现：github.com/ErvinXie/AaaS（协议、服务端与客户端）

服务端不能凭模型输出宣布完成；最终状态由数据和设备所在的一端确认。

边界画在哪里：只看 Context 归谁

两种都成立，分界的判据是同一条：Context 归谁。

完整 Agent 服务

- Runtime、会话和工作环境一起托管
- 客户端负责发起、观察和继续
- Context 随服务端 Agent 一起维护
- 例：Dots、Muse、cdxm server

同一个问题

Context 归谁?

Agent Runtime 服务

- 服务端只负责推理与推进
- 客户端提供 Context 和工具
- 业务事实由数据所在端确认
- 例：SuperMe（超我）

服务端内部



Runtime ↔ Model → 服务内闭环

服务端 ↔ 用户侧



Runtime ↔ Client → 按需交换

不是 AaaS 的唯一标准，而是两种不同的服务边界；媒体按需、限量、带版本地提供。

cdxm server：手机负责接入，服务器负责运行

第一次解耦的实现是 meta-agent：用仓库承载 Context；cdxm server 则让它在自己的服务器上跑——账号、项目和历史会话都在自己手里。



会话留在服务器上

五个会话各有独立的工作目录，关掉浏览器仍在继续

账号与身份自持

codex / pi 两套身份，模型走自己的路由

手机只是接入端

换台设备回来，还是同一个会话

浏览器关掉不等于 Agent 停止；换台设备回来，会话还在。

一句话，延续一项生活计划

第二次解耦的实现是 AaaS（Runtime 服务化，github.com/ErvinXie/AaaS）；SuperMe 是它的第一个应用——价值由用户确认，估计值、用户修改值和已确认值分开保存。

SCREEN RECORDING



11 分 48 秒 · 原速播放，可拖动进度条
（地址与手机号已打码）

USER REQUEST

按之前的目标，把下周饮食和运动计划调整一下。

读取相关 Context

理解这次变化

生成候选方案

客户端确认保存

不再依赖某一次运行，也不再依赖某一台机器

第一次解耦把工作状态从一次运行里拿出来；第二次解耦把运行位置从用户的数据、应用和设备里拿出来。

01 · FIRST DECOUPLING

工作状态从一次运行中解耦出来

02 · SECOND DECOUPLING

运行位置从用户的数据、应用与设备中解耦出来

RESULT

更多任务变得可以判断：可恢复、可核对、可换模型

开源实现：github.com/Approaching-AI/meta-agent

两次解耦不是把 Agent 变强，而是让更多任务变得可以判断。